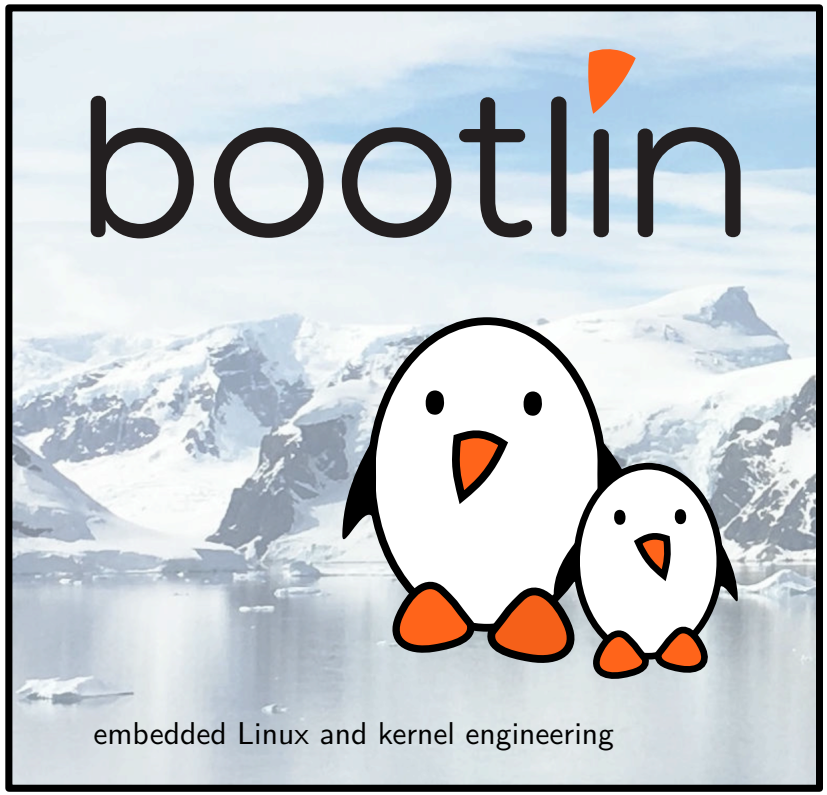
Using Yocto Project - basics



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Environment setup



Environment setup

- All Yocto source files are left unchanged when building a custom image.
- Specific configuration files and build repositories are stored in a separate build directory.
- A script, `oe-init-build-env` , is provided to set up the build directory and the environment variables (needed to be able to use the `bitbake` command for example).



oe-init-build-env

- Modifies the environment: has to be sourced!
- Adds environment variables, used by the build engine.
- Allows you to use commands provided in openembedded-core.
- `source ./oe-init-build-env [builddir]`
- Sets up a basic build directory, named `builddir` if it is not found. If not provided, the default name is `build`.



The initial `build/` directory

- The `oe-init-build-env` script creates the `build` directory with only one subdirectory in it:
  - **conf/** Configuration files. Image specific and layer configuration



Exported environment variables

BUILDDIR Absolute path of the build directory.

PATH Contains the directories where executable programs are located. Absolute paths to `scripts/` and `bitbake/bin/` are prepended.



Available commands

bitbake	The main build engine command. Used to perform tasks on available recipes (download, configure, compile...).
bitbake-*	Various specific commands related to the BitBake build engine.



Configuring the build system



The `build/conf/` directory

- The `conf/` directory in the `build` one holds two mandatory build-specific configuration files:

  bblayers.conf

  Explicitly list the layers to use.

  local.conf

  Set up the configuration variables relative to the current user for the build.
  Configuration variables can be overridden there.

- Additional optional configuration files can be used:

  site.conf

  Similar to `local.conf` but intended to be used for site-specific settings, such as network mirrors and CPU/memory resource usage limits.

  toolcfg.conf

  Used by tools to store build configuration, it should not be modified manually.



Configuring the build

The `conf/local.conf` configuration file holds local user configuration variables:

- BB_NUMBER_THREADS: How many tasks BitBake should perform in parallel. Defaults to the number of CPU threads on the system.
- PARALLEL_MAKE: How many processes should be used when compiling. Defaults to the number of CPU threads on the system.
- MACHINE: The machine the target is built for, e.g. `beaglebone`.



Building an image



Compilation

- The compilation is handled by the BitBake *build engine*.
- Usage: `bitbake [options] [recipename/target ...]`
- To build a target: `bitbake [target]`
- Building a minimal image: `bitbake core-image-minimal`
  - This will run a full build for the selected target.
- The `oe-init-build-env` script lists some more example targets



The build/ directory after the build 1/2

conf/	Configuration files, as before, not touched by the build.
downloads/	Downloaded upstream tarballs of the recipes used in the builds.
sstate-cache/	Shared state cache. Used by all builds.
tmp/	Holds all the build system outputs.



The build/ directory after the build 2/2

tmp/work/

Set of specific work directories, split by architecture. They are used to unpack, configure and build the packages. Contains the patched sources, generated objects and logs.

tmp/sysroots/

Shared libraries and headers used to compile applications for the target but also for the host.

tmp/deploy/

Final output of the build.

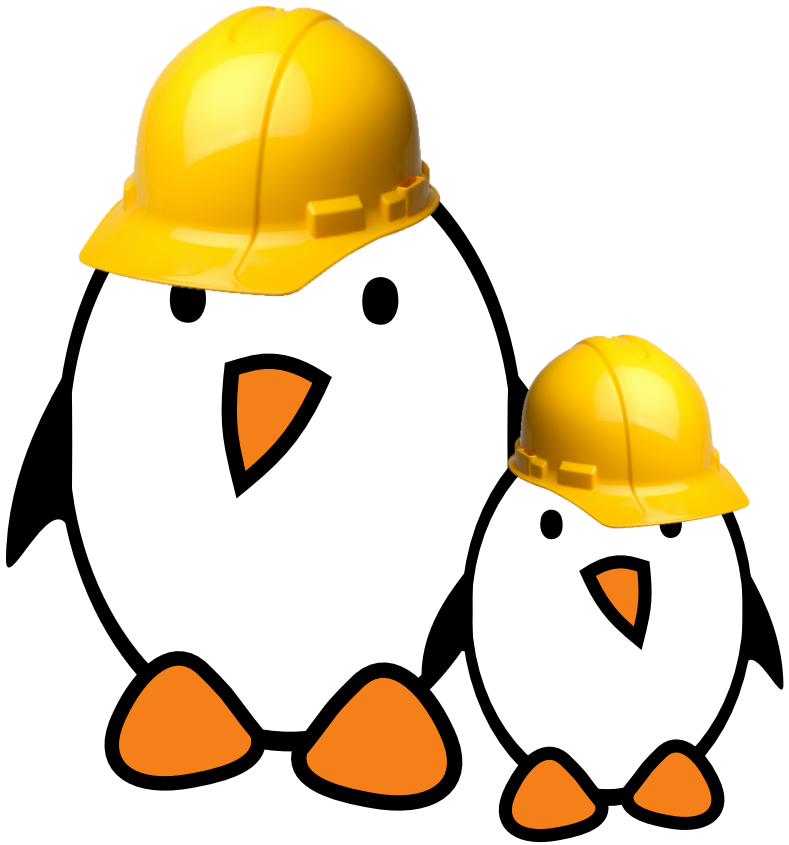
tmp/deploy/images/

Contains the complete images built by the OpenEmbedded build system. These images are used to flash the target.

tmp/buildstats/

Build statistics for all packages built (CPU usage, elapsed time, host, timestamps...).

Practical lab - First Yocto build



- Download the sources
- Set up the environment
- Configure the build
- Build an image